Ferric FX: a web-based audio effects processor in p5.js

(words: 499)

Link: https://hub.labs.coursera.org/connect/sharedwtihvxaf

David Cardozo

5 July 2026

The application

Ferric FX is a web audio application built with p5.js and its p5.sound library. A pre-recorded sound file plays through a fixed processing chain (low-pass filter, waveshaper distortion, dynamic compressor, reverb and master volume); the result is sent to the output device, recorded to a WAV file, or saved into a sounds library.

The chain is built once at start-up in `audioEngine.js`. Every p5.sound effect auto-connects to the speakers, so each node is first `disconnect()`ed and then wired in series (`source → p5.LowPass → p5.Distortion → p5.Compressor → p5.Reverb → p5.Gain → output`), the fixed signal flow shown in Figure 2. Two `p5.FFT` analysers tap the chain's head and tail to draw the *Spectrum In* and *Spectrum Out* displays, so every edit is immediately visible. Transport buttons (play/pause, stop, loop, skip to start) drive the current `p5.SoundFile`.



Figure 1: The interface: transport and REC, signal-flow strip, live in/out spectra, per-stage editor, sounds library.



Figure 2: Internal signal flow. A single fixed chain carries the source through the five effect stages to the output device; `audioEngine.js` builds it once and thereafter only pushes parameter changes into the live nodes.

The effects

Low-pass filter (`p5.LowPass`, via `freq()` and `res()`) attenuates everything above its cutoff at 12 dB/oct, while resonance emphasises the corner frequency. Figure 3 shows a broadband input reduced to its lowest bands by a 400 Hz cutoff.



Figure 3: Low-pass at 400 Hz, fully wet, other stages bypassed: input energy above the cutoff vanishes from the output.

Waveshaper distortion (`p5.Distortion`, `set(amount, oversample)`) pushes the signal through a nonlinear transfer curve. Unlike a filter it creates new spectral content: harmonics of the input partials. In Figure 4 the same filtered signal is driven hard and energy reappears *above* the cutoff, generated entirely by the distortion. Oversampling (2×/4×) tames the aliasing.

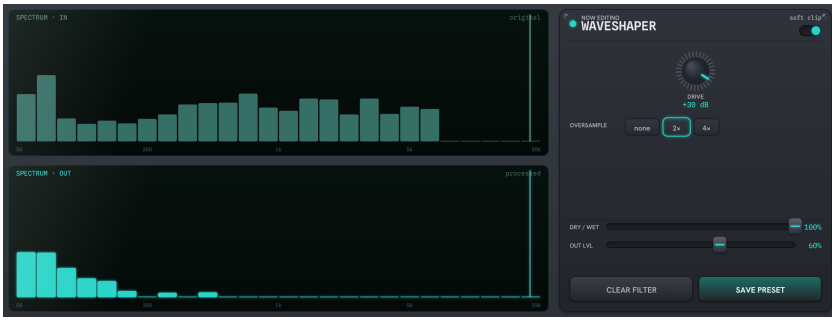


Figure 4: The waveshaper re-enabled after the same 400 Hz filter: harmonics reappear above the cutoff.

Dynamic compressor (`p5.Compressor`: threshold, ratio, knee, attack, release) reduces dynamic range, adding no new frequencies; magnitudes stay more constant over time, raising average loudness.

Reverb (`p5.Reverb`, `set(duration, decay, reverse)`) convolves the signal with a decaying impulse response, smearing energy in time and filling the gaps between transients: a denser, sustained spectrum.

Master volume (`p5.Gain`) scales the mix uniformly, so the spectrum keeps its shape at a different level. Each stage also has DRY/WET and OUT LVL faders; bypass is implemented as `drywet(0); amp(1)`, passing the dry signal at unity so the graph never needs rewiring.

Two enhanced filter options

A *feedback delay* (`p5.Delay`) would add echoes; at short delay times the delayed copy interferes with the direct signal, carving comb-filter notches spaced at 1/delay Hz across the spectrum.[1] A *multi-band equaliser* (`p5.EQ`) would allow surgical boosts and cuts at chosen frequencies rather than the low-pass's single downward slope.[2] Both are p5.sound nodes that could be patched into the chain unchanged.

[1] `p5.Delay` reference: https://archive.p5js.org/reference/#/p5.Delay. On the comb filtering produced by short feedback delays, see J. O. Smith III, *Physical Audio Signal Processing*, W3K Publishing, 2010, §"Feedback Comb Filters".

[2] `p5.EQ` reference: https://archive.p5js.org/reference/#/p5.EQ. Equaliser and filter effect design is treated in J. D. Reiss and A. P. McPherson, *Audio Effects: Theory, Implementation and Application*, CRC Press, 2015.

The sounds library

Six pre-recorded WAVs load into the library panel at start-up. Selecting a card loads it into the chain; EDIT opens its stored effect configuration in the editor, CLR resets it to a dry source, and DEL removes it. REC records the processed master bus with `p5.SoundRecorder`; stopping opens the capture dialog (Figure 5) where the take can be auditioned, renamed, exported as a WAV via `save()`, or added to the library with its effect settings baked in.

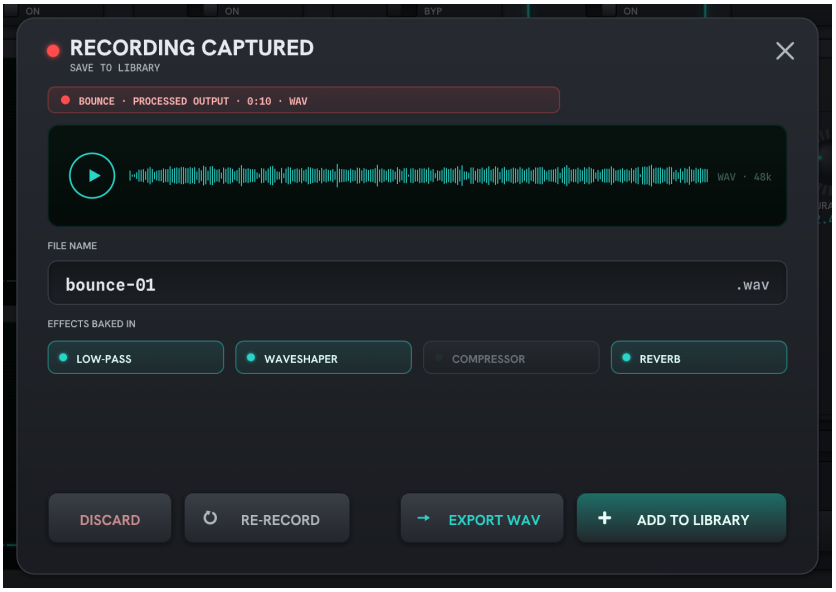


Figure 5: The capture dialog: waveform preview, file name, the effects baked into the take, export/save actions.

The new card lists its stored effects (`LPF · WSH · RVB`, Figure 6). SAVE PRESET persists the live settings; the library index lives in `localStorage` and recorded or imported audio in IndexedDB, so sounds *and* their effect configurations survive a reload to be re-edited, cleared or deleted later.



Figure 6: The library after saving: `bounce-01` carries its baked-in effect chain.

Appendix: constructing the chain

How `audioEngine.js` builds the graph of Figure 2 once at start-up (abridged):

```
nodes = {
  lowpass:    new p5.LowPass(),
  waveshaper: new p5.Distortion(),
  compressor: new p5.Compressor(),
  reverb:     new p5.Reverb(),
  masterGain: new p5.Gain(),
};
// undo each effect's default route
// to the speakers
[nodes.lowpass, nodes.waveshaper,
 nodes.compressor, nodes.reverb]
  .forEach((n) => n.disconnect());
// wire the fixed series chain
nodes.lowpass.connect(nodes.waveshaper.input);
nodes.waveshaper.connect(nodes.compressor.input);
nodes.compressor.connect(nodes.reverb.input);
nodes.reverb.connect(nodes.masterGain.input);
nodes.masterGain.connect(); // -> output device

fftIn = new p5.FFT();  // Spectrum In: the source
fftOut = new p5.FFT(); // Spectrum Out: post-FX
fftOut.setInput(nodes.masterGain);
recorder = new p5.SoundRecorder();
recorder.setInput(nodes.masterGain); // REC taps here
```

Exercise 2: Audio Visualisation with Meyda and a p5.speech Voice Controller

(words: 499)

Link: https://hub.labs.coursera.org/connect/sharedphwuxhyd

5 July 2026

Both applications are p5.js sketches built on one idea: measured properties of the audio, not hand-animated timelines, drive every visual variable.[1] This report describes what is mapped and why, from the standpoint of visual impact, and shows the voice controller working. All screenshots are captured live from the running applications.

[1] Meyda: audio feature extraction for the Web Audio API, https://meyda.js.org. p5.speech: Web Speech API wrappers for p5.js, https://idmnyu.github.io/p5.js-speech/.

Task 2.1: Sonifying the CAPTCHA

The audio CAPTCHA presents three word–digit pairs using p5.speech. For each word, the system randomly selects a voice, pitch, and speaking rate, so repeated attempts do not sound identical. A p5.sound masking layer runs underneath the speech, combining pink noise band-passed around the speech range near 1.4 kHz with a low brown-noise rumble. The mask is also amplitude-modulated, which makes it harder for a bot recording the browser tab to subtract the background cleanly.



Figure 1: Task 2.1 mid-challenge. The LCD scope pulses with the spoken word; the trace is the band-passed masking bed, peak-normalised per frame and scaled by the word-synced envelope.

Because the browser speech-synthesis API does not expose the generated audio buffer, the spoken voice cannot be analysed directly in the sketch. Instead, the oscilloscope visualises the masking layer, which is the actual signal available in the p5.sound graph. Each p5.FFT waveform frame is normalised to its own peak and then scaled by an envelope

that rises while a word is being spoken. As a result, the LCD display appears to pulse in sync with the speech, as shown in Figure 1.

Task 2.2: Feature selection and mapping

Feature selection was measured, not guessed: an offline pass with librosa over 512-sample frames (Meyda's exact buffer size) computed how much each candidate feature actually moves in each sound.[2] The rule: map the features with the largest relative swing, because a feature that barely changes produces a static, invisible mapping. Each sound then receives a visual form that suits its measured character (table below, Figures 2–4).

[2] Coefficient of variation, CV = std/mean, per feature and sound; the full statistics are in the appendix.

Sound (measured character)	Meyda features	Mapping, from a visual-impact standpoint
1: dark (centroid ≈ 810 Hz), tonal (flatness ≈ 0), sparse events (flux CV 0.62)	rms, spectralFlux, spectralCentroid	"Ripple pond": rings breathe with loudness; each onset launches a travelling ripple; hue drifts inside a dark blue–violet band, preserving the sound's identity
2: volatile; rms CV 1.15, centroid CV 1.26 across 10.9 kHz, flatness spans 0–1	spectralCentroid, rms, spectralFlatness	"Particle burst field": colour runs dark cyan to hot magenta with brightness; emission rate and size explode with loudness; flatness turns laminar streams into turbulence
3: bright (centroid ≈ 4 kHz), high-pitched (zcr 0.16), rhythmic (flux CV 0.81)	rms, spectralFlux, zcr	"Spark shower": onsets fire firework bursts; rms sets spark size and shimmer density; zcr pins the amber-to-green hue signature

All features are normalised to 0–1 and exponentially smoothed so the mappings stay stable and comparable across sounds. Meyda's streaming analyzer cannot compute spectralFlux (it lacks the previous frame), so flux is derived from consecutive amplitude spectra as rectified rising energy with an adaptive peak; a value 1.8× above its running baseline is treated as an onset. Together the mappings exercise the brief's visual variables: number of shapes, size, fill colour, opacity, background colour and rotation.

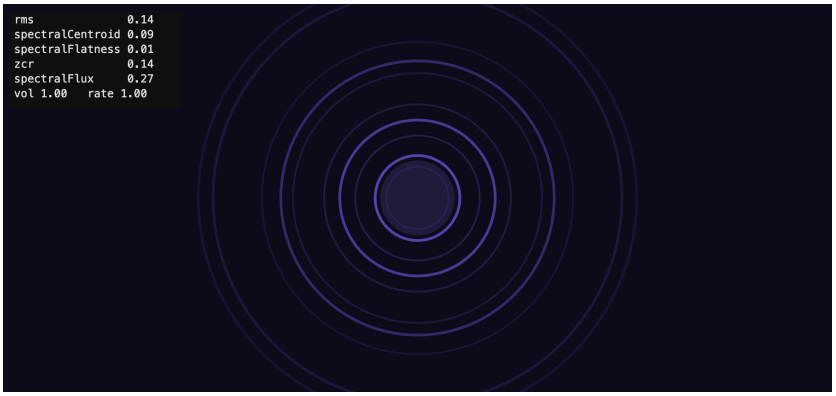


Figure 2: Sound 1, ripple pond. Rings breathe with rms; a flux onset has launched travelling ripples; the live feature readout sits top-left.



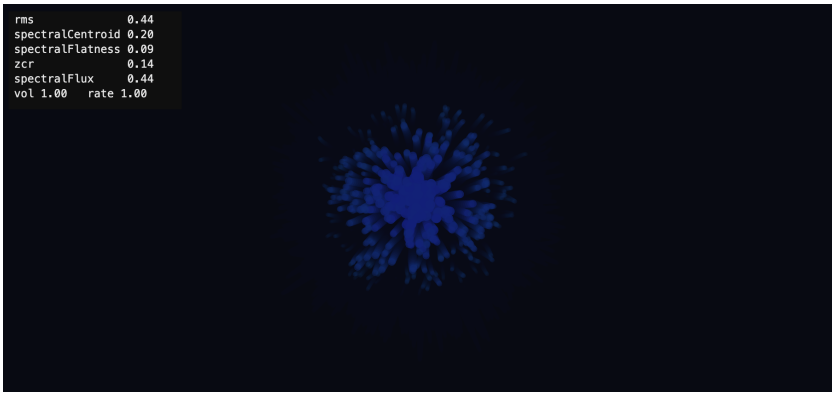


Figure 3: Sound 2 at both extremes: quiet dark-cyan wisps (top, rms 0.25) versus a loud burst of 375 particles (bottom, rms 0.44). One mapping, radically different frames: exactly why the high-CV features were chosen.

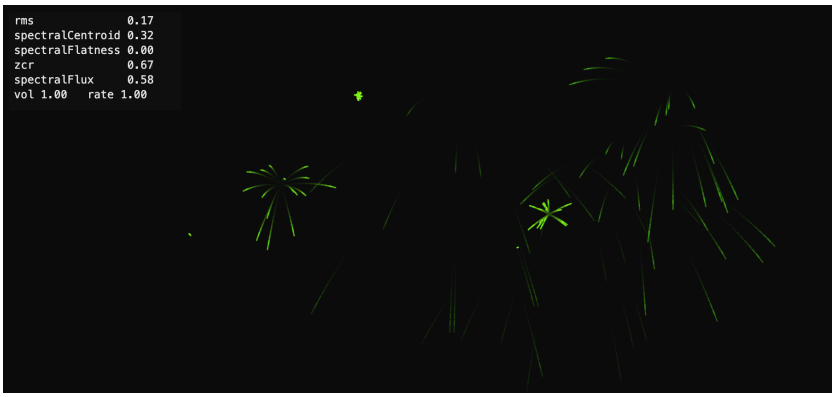


Figure 4: Sound 3, spark shower. Flux onsets fire firework bursts, rms sets spark size, and zcr (0.67 here) pins the yellow-green hue of this bright, high-pitched sound.

Task 2.3: Voice controller

The voice controller runs p5.SpeechRec in continuous mode; recognition engines stop on silence, so an `onEnd` handler restarts listening while the toggle is on. Each utterance is scanned word by word and the latest keyword per category wins, with homophones accepted where recognisers commonly mishear ("sound *to/too/two*"). The required commands (five background colours and four shapes) act as an override layer on every visual, and the extended set covers transport, volume, playback rate, figure scale and trail effects:

Category	Spoken words	Effect
Background	black, white, red, blue, green	overrides the visual's native backdrop
Shape	square, triangle, circle, pentagon	re-renders every generated figure
Transport	play, stop, next, "sound one…four"	track switching, homophone-tolerant
Volume / speed	louder, quieter, faster, slower	± 0.2 volume; $\times 1.25$ playback rate
Effects	bigger, smaller, trails, clean, invert, reset	figure scale, trail overpaint, brightness flip, clear all

Applied to the *Kalte Ohren* remix, visualised as a figure field whose size pulses with rms (CV 0.54), spawns bursts on flux onsets (CV 0.63) and colours figures by centroid (CV 0.50), the recognised phrase "green triangle" repaints the background and re-renders every figure within one frame (Figure 5). A chip beside the transport echoes the last understood command, and a denied microphone cleanly shuts the listening loop down instead of retrying forever.

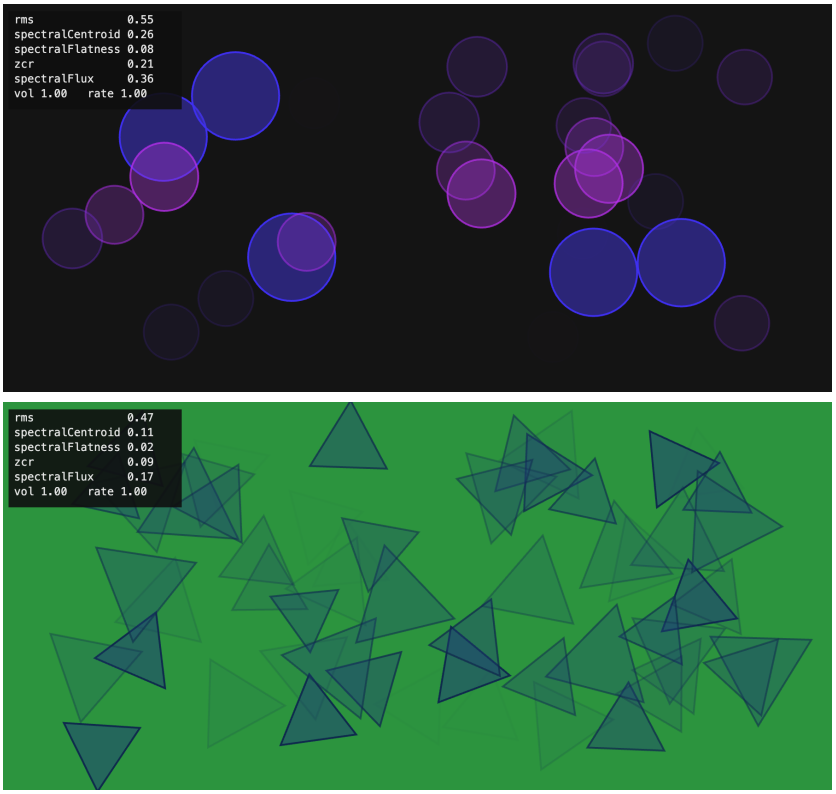


Figure 5: Task 2.3 on the Kalte Ohren remix. Native look (top); one utterance later ("green triangle") the background colour and figure shape are overridden (bottom).

Appendix

A. Feature statistics (librosa, frame = hop = 512)

Coefficient of variation (std/mean) per feature; the mean rows give each sound's absolute character. Flatness CVs are inflated by near-zero means and were used only as a tonal-vs-noisy indicator.

	Sound 1	Sound 2	Sound 3	Remix
duration (s)	5.3	14.6	14.3	280.0
rms CV	0.46	1.15	0.75	0.54
centroid CV	0.28	1.26	0.12	0.50
flatness CV	4.52	4.14	2.73	21.95
zcr CV	0.26	1.13	0.19	0.88
flux CV	0.62	1.35	0.81	0.63
mean rms	0.052	0.061	0.057	0.202
mean centroid (Hz)	810	1786	3962	3164
mean zcr	0.028	0.028	0.159	0.059

B. Code excerpts

The statistics come from `analyze.py` (librosa, frame and hop of 512 samples to mirror Meyda's `bufferSize`), run with `uv run analyze.py`. The excerpts below are from `sketch.js`.

Deriving spectralFlux from consecutive amplitude spectra, inside the Meyda analyzer callback:

```
let flux = 0;
for (let i = 0; i < spec.length; i++) {
  const d = spec[i] - prevSpectrum[i];
  if (d > 0) flux += d; // rising energy only
}
```

Adaptive flux normalisation and onset detection; a spike spawns a ripple, spark burst or figure burst depending on the sound:

```
fluxMax = Math.max(fluxMax * 0.999, rawFlux);
const fluxN = clamp01(rawFlux / fluxMax);
fluxBaseline = lerp(fluxBaseline, fluxN, 0.05);
const isSpike = fluxN > 0.18 &&
  fluxN > fluxBaseline * 1.8 &&
  spawnCooldown === 0;
```

Scanning each recognised utterance for command keywords; the latest word in a category wins:

```
const words = str.toLowerCase()
  .replace(/[^a-z0-9\s]/g, " ")
  .split(/\s+/);
for (const w of words) {
  if (Object.hasOwn(VOICE_COLORS, w))
    voiceState.bg = w;
  else if (VOICE_SHAPES.includes(w))
    voiceState.shape = w;
  // ...transport, volume, effects...
}
```

Audio steganography: ultrasonic AM detection, key-based LSB embedding, and a diffusion-based approach

(words: 695)

Link: https://hub.labs.coursera.org/connect/sharedfrwficmn

5 July 2026

Overview

Tasks 3.1 and 3.2 are implemented as Python notebooks: one detects and decodes a code hidden in an ultrasonic band, the other embeds and recovers a text message with a key-based least-significant-bit (LSB) scheme. Task 3.3 reviews a recent diffusion-based method and compares it with LSB.

Task 3.1: Analysis of the suspicious audio

The problem. Four WAV files may carry a four-number code that has been amplitude-modulated (AM) onto an *ultrasonic* carrier and mixed into an ordinary recording, so nothing sounds wrong on normal playback.

Detection. The container headers already narrow the field: the three 48 kHz files have Nyquist headroom (24 kHz) for ultrasound, whereas the 44.1 kHz file barely does. For each file, the detector measures the fraction of spectral energy at or above 18 kHz, a region where natural recordings hold almost nothing. `Ex3_sound4.wav` keeps **10.86 %** of its energy there against 0.006–4.25 % for the others, peaking at **19,001 Hz** (Figure 1). That is the carrier.

```
In [5]: ULTRA = 18000.0
        rows = []
        for name, (sr, x) in signals.items():
            mag = np.abs(np.fft.rfft(x * np.hanning(len(x))))
            freq = np.fft.rfftfreq(len(x), 1/sr)
            m = (freq >= ULTRA) & (freq < sr/2 - 1.0)
            frac = mag[m].sum() / (mag.sum() + 1e-12)
            carrier = freq[m][np.argmax(mag[m])]
            rows.append((name, frac*100, carrier))

        print(f"{"file":<18}{"ultrasonic energy":>20}{"peak freq":>12}")
        print("-" * 50)
        for name, pct, cf in rows:
            flag = "  <== SUSPICIOUS" if pct > 8 else ""
            print(f"{name:<18}{pct:>18.3f}%{cf:>10.0f}Hz{flag}")
```

```
file                ultrasonic energy   peak freq
--------------------------------------------------
Ex3_sound1.wav                 4.246%     18523Hz
Ex3_sound2.wav                 0.014%     18638Hz
Ex3_sound3.wav                 0.006%     21074Hz
Ex3_sound4.wav                10.863%     19001Hz  <== SUSPICIOUS
```

Figure 1: Detection evidence: ultrasonic energy and peak frequency per file; `Ex3_sound4.wav` is flagged automatically.

Diagnosis. Zooming into that band shows a central carrier line flanked by *symmetric sidebands* (Figure 2), the textbook fingerprint of amplitude modulation: AM shifts the message spectrum up around the

carrier and mirrors it on both sides.

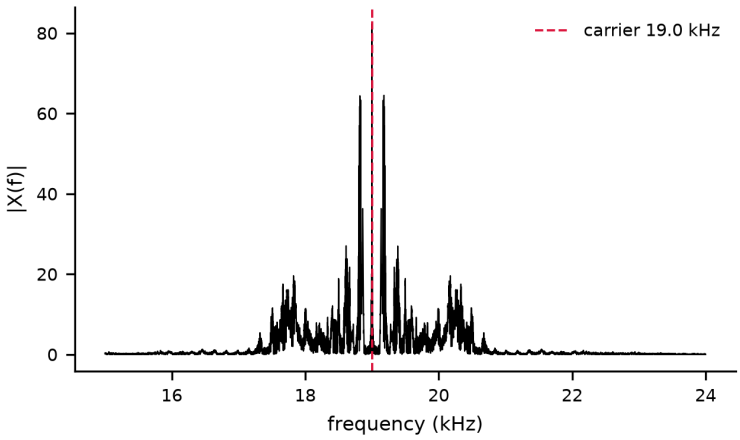


Figure 2: The ultrasonic band of `Ex3_sound4.wav`: a carrier at 19 kHz with symmetric sidebands, the AM fingerprint.

Decoding. The decoder reverses the modulation in three steps: a band-pass filter isolates the carrier; envelope detection via the analytic (Hilbert) signal strips the carrier oscillation and leaves the message; a low-pass filter returns it to baseband. The recovered signal shows voiced-speech harmonics (Figure 3, right) and, played aloud, speaks the code **1891**.

Task 3.2: Embedding a hidden message

Task 3.2 embeds *"It always seems impossible until it's done"* into `Ex3_sound5.wav` and recovers it, improving on plain LSB in three ways. A single integer **key** seeds a pseudo-random permutation of the sample indices, so the message bits are scattered across *non-consecutive* samples, and only the key has to be shared, not the list of positions. Each chosen sample carries **two** least-significant bits, doubling capacity. A 32-bit length **header** is embedded first, so extraction is self-contained: the receiver reads the header, learns the message length, and stops on its own.

Results. The message occupies **184 of 407,280 samples (0.045 %** of capacity); only **132** samples change, each by at most **±3** out of 65,536 (Figure 4), far below the threshold of hearing. The altered samples are sparse and scattered, and the cover and stego spectra coincide, leaving no visible trace (Figure 5). Extraction with the correct key reproduces the message *exactly*; a wrong key selects the wrong samples and decoding fails on invalid UTF-8 (Figure 6), confirming that the key genuinely protects the payload.

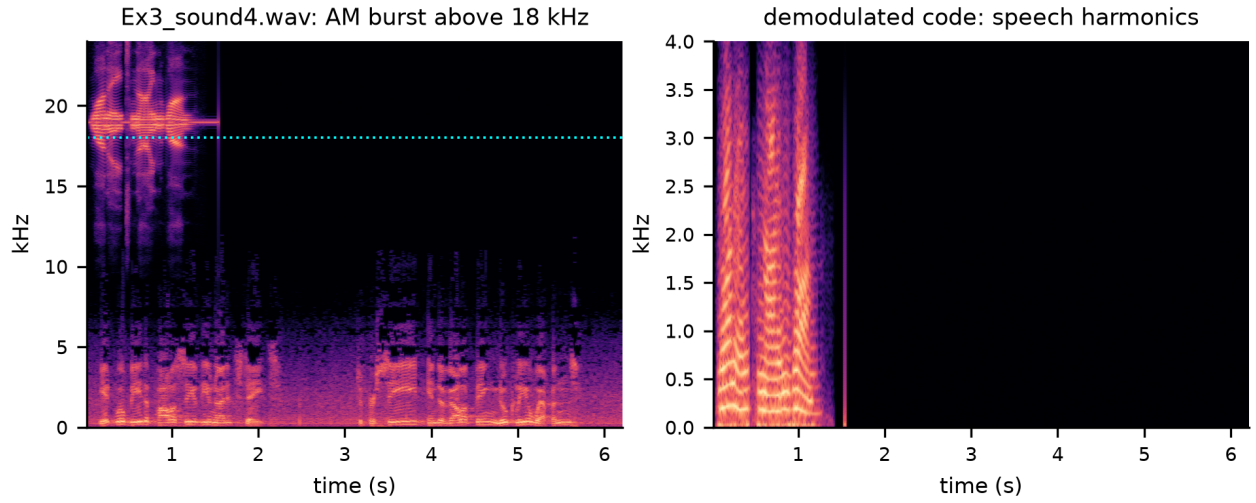


Figure 3. Left: the stego spectrogram; the AM burst sits above the 18 kHz line (cyan) and lasts only the first ≈1.6 s. Right: after band-pass, Hilbert-envelope and low-pass demodulation, the code returns to the audible band as clear voiced-speech harmonics.

```
In [5]: stats = embed_message(INPUT_WAV, OUTPUT_WAV, MESSAGE, KEY, N_LSB)

        print(f'Secret message : {MESSAGE!r}')
        print(f'Stego key      : {KEY}')
        print(f'Bits per sample: {N_LSB}')
        print(f'Samples used   : {stats["slots"]} of {stats["capacity"]} '
              f'({100 * stats["slots"] / stats["capacity"]:.3f}% of capacity)')
        print(f'Samples changed: {stats["changed"]} (max amplitude change +/-{stats["max_delta"]})')
        print(f'Written to     : {OUTPUT_WAV}')

Secret message : "It always seems impossible until it's done"
Stego key      : 20240703
Bits per sample: 2
Samples used   : 184 of 407280 (0.045% of capacity)
Samples changed: 132 (max amplitude change +/-3)
Written to     : out/Ex3_sound5_embedded.wav
```

Figure 4: Embedding evidence: 184 of 407,280 samples used, 132 changed, each by at most ±3.

Task 3.3: A state-of-the-art approach, diffusion-based steganography

Xi et al. (2025) propose a **diffusion-based** audio steganography model.[1]

Characteristics and implementation. Two coupled modules share one denoising-diffusion backbone: a U-Net trained to predict, and therefore remove, the noise added to audio at each diffusion step. The *embedding* module folds the secret bits into the model's starting noise (its latent) and then denoises that latent into natural-sounding stego audio. A psychoacoustic attention map, computed from an STFT, steers the noise away from the low frequencies the ear is most sensitive to. Because the sampling schedule is deterministic (DDIM), the chain is invertible: the *extraction* module simply runs it backwards, from stego audio to latent, and reads the bits out of the latent's DCT coefficient signs, exactly on a clean channel. The framework can even *generate* the cover audio outright, needing no natural carrier. The appendix walks through this pipeline in plain terms.

[1] J. Xi, Z. Xia, W. Zhang, Y. Xie and L. Zhao, "Diffusion-Based Model for Audio Steganography," *Electronics* **14**(20):4019, 2025. https://doi.org/10.3390/electronics14204019.

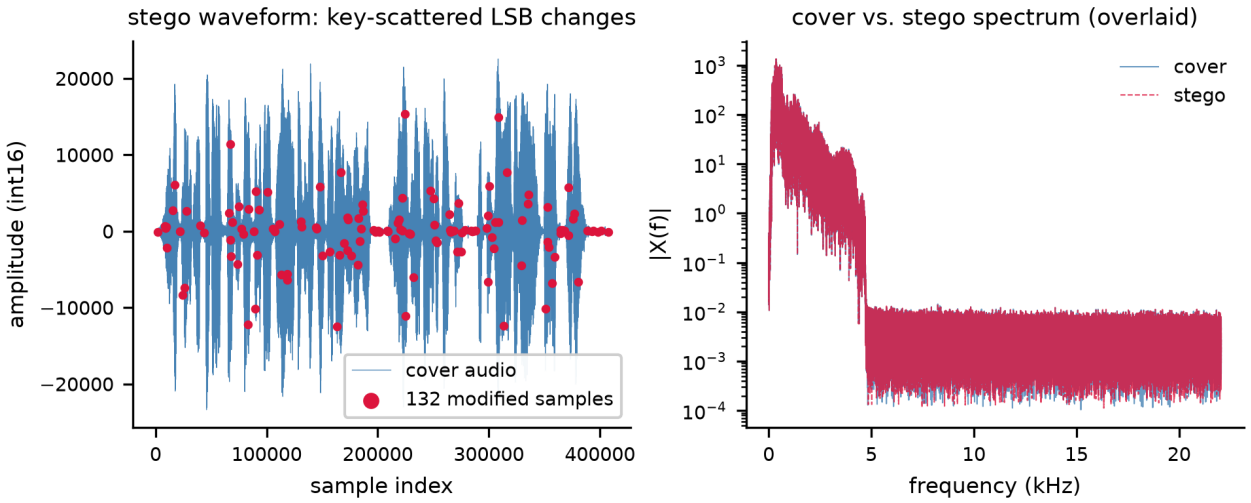


Figure 5. Left: the cover waveform with the 132 key-selected message samples marked in red, sparse and scattered. Right: cover and stego magnitude spectra overlaid; the curves coincide, so the embedding leaves no visible spectral trace.

```
In [6]: recovered = extract_message(OUTPUT_WAV, KEY, N_LSB)
        print(f'Extracted message: {recovered!r}')
        print(f'Matches original : {recovered == MESSAGE}')

        # A wrong key selects the wrong samples: the recovered bytes are noise and
        # usually aren't even valid UTF-8, so decoding fails outright.
        try:
            wrong = extract_message(OUTPUT_WAV, KEY + 1, N_LSB)
            print(f'\nWith wrong key   : {wrong[:30]!r} ...  (matches: {wrong == MESSAGE})')
        except UnicodeDecodeError:
            print('\nWith wrong key   : decoding failed (invalid UTF-8) — message not recoverable')

Extracted message: "It always seems impossible until it's done"
Matches original : True

With wrong key   : decoding failed (invalid UTF-8) — message not recoverable
```

Figure 6: Extraction evidence: the correct key recovers the message exactly; a wrong key fails to decode.

Benefits. The authors report near-transparent quality (PESQ 4.42, SNR 83 dB) and, critically, deep-learning steganalyzers detect the stego audio at only **about 47–62 %**, close to the 50 % chance line, versus **about 76 %** for LSB matching. The method also survives MP3 compression, added noise and resampling.

Comparison with LSB. Table 1 summarises the trade-offs.

Capacity favours LSB, which can claim every cover sample; detectability favours diffusion, and the Task 3.2 key cannot close that gap. Scattering hides *where* the bits sit, but the statistical trace of bit replacement survives, and a steganalyzer needs only the trace, never the key: the key protects confidentiality, not covertness. Diffusion never leaves the trace, because its stego audio is generated from a learned model of natural audio, not edited after the fact. LSB therefore suits benign channels that need simplicity and bit-exact recovery; against steganalysis or lossy transmission, diffusion is the stronger design.

	Key-based LSB (Task 3.2)	**Diffusion (Xi et al.)**
Embedding domain	time (raw sample bits)	learned deep features
Capacity	2 bits per selected sample	0.1–0.5 bits per sample tested
Detectability	high (LSB ≈76%)	very low (≈47–62%)
Robustness	fragile to resampling	survives noise / MP3
Shared secret	one integer key	a trained network
Cost	negligible, CPU	GPU training + inference
Clean recovery	bit-exact, free	exact but costly

Table 1. The key-based LSB scheme of Task 3.2 and the diffusion model of Xi et al., compared.

Appendix: the diffusion pipeline in plain terms

A diffusion model learns to turn noise into realistic data. During training, clean audio is corrupted step by step with small amounts of Gaussian noise, and a neural network (a U-Net) learns to undo one step of that corruption: given a noisy clip and the step number, it predicts the noise that was added. Once trained, the network can start from pure noise and denoise its way, step by step, to audio that sounds natural. Xi et al. build their steganography on three consequences of this design.

Hiding. The secret bits are not written into an existing file sample by sample. Instead they are folded into the starting noise, called the latent: each bit sets the sign of one DCT coefficient of that latent (positive for a one, negative for a zero). Running the denoising process on this shaped latent produces the stego audio.

Recovery. The generator uses the deterministic DDIM variant: the random term in each denoising step is set to zero, which turns the sampling chain into an ordinary differential equation solved with small Euler steps. The same latent therefore always produces the same audio, and the chain can be run in reverse: the receiver, holding the same trained network, walks the stego audio back to its latent, takes the DCT, and reads the signs. On a clean channel, this recovery is exact.

Imperceptibility. While noise is being injected, an attention weight decides where it may go. The audio is mapped to a time-frequency grid with a short-time Fourier transform, a psychoacoustic model scores each bin by how sensitive the ear is to it, and the weights make sure the perceptually critical low frequencies are disturbed least.

The security argument follows from the construction: the stego audio is not a modified natural recording but a sample drawn from the model's learned distribution of natural audio, so a steganalyzer finds no processing artefacts to latch onto. The price is practical: sender and receiver must share the trained network itself rather than a small key, embedding and extraction each cost many network evaluations, and noisy channels raise the bit error rate to roughly 5–6 %.

An Offline Multilingual Speech Interface for an Airport Virtual Assistant

Exercise 4 (words: 800)
Link: https://hub.labs.coursera.org/connect/sharedfrwficmn

6 July 2026

This report documents the prototype speech interface built for the client's airport virtual assistant (Task 4.1) and closes with a comparison against an ASR system deployed in the e-commerce industry (Task 4.2). The prototype is a Python application that recognises short travel phrases in English, Spanish and Italian entirely on the local machine; is evaluated on the client's fifteen recorded phrases plus two recordings made by the author; and reports a word error rate (WER) for every file.

The ASR package and its configuration

The module's suggested engine, Mozilla DeepSpeech 0.9.3, has been archived and unmaintained since 2021[1], which makes it a poor foundation for a kiosk the client must operate for years. **sherpa-onnx** 1.9.30[2], the deployment runtime of the k2/icefall speech ecosystem, was therefore selected: it is actively maintained, runs entirely offline as the client requires, and executes neural transducer models[3] through `onnxruntime` 1.14.1 on a plain CPU. Both packages are pinned by version and installed under Python 3.7, matching the module's Coursera environment (Figure 1).

Language selection is a dictionary that maps each of the three languages to its own recognizer, each built from a model bundle on local disk: a streaming Zipformer transducer[4] for English and Kroko `kroko_64l` transducers for Spanish and Italian. In every case the int8-quantised weights are kept because they are smaller and faster on kiosk-class hardware. Each recognizer is created with `OnlineRecognizer.from_transducer(...)` configured for greedy-search decoding, the CPU provider and two threads.

Handling the noisy environment

An airport concourse combines low-frequency rumble (ventilation, engines, rolling luggage) with speakers standing at unpredictable distances from the microphone. The application therefore cleans every utterance with a three-stage `preprocess()` chain before decoding. First, an order-4 Butterworth high-pass filter at 80 Hz removes the rumble band, which sits below the fundamental frequency of adult speech; it is applied

[1] Mozilla, *DeepSpeech* 0.9.3 (2020), `github.com/mozilla/DeepSpeech`; repository archived, last release December 2020.

[2] k2-fsa project, *sherpa-onnx* v1.9.30, `github.com/k2-fsa/sherpa-onnx`.

[3] A. Graves, "Sequence transduction with recurrent neural networks," ICML Workshop on Representation Learning, 2012. arXiv:1211.3711.

[4] Z. Yao et al., "Zipformer: A faster and better encoder for automatic speech recognition," Proc. ICLR, 2024. arXiv:2310.11230.

```
!python -m pip install sherpa-onnx==1.9.30 onnxruntime==1.14.1

English: already extracted at models/sherpa-onnx-streaming-zipformer-en-2023-06-26
ES (Kroko kroko_64l): downloading...
  skip (exists): models/kroko-es/encoder.int8.onnx
  skip (exists): models/kroko-es/decoder.int8.onnx
  skip (exists): models/kroko-es/joiner.int8.onnx
  skip (exists): models/kroko-es/tokens.txt
ES: ready at models/kroko-es
IT (Kroko kroko_64l): downloading...
  skip (exists): models/kroko-it/encoder.int8.onnx
  skip (exists): models/kroko-it/decoder.int8.onnx
  skip (exists): models/kroko-it/joiner.int8.onnx
  skip (exists): models/kroko-it/tokens.txt
IT: ready at models/kroko-it
```

Figure 1: Configuration evidence from the notebook: the version-pinned installation cell and the three offline model bundles resolving from local disk.

forwards and backwards (`sosfiltfilt`), so the filtering is zero-phase and does not smear phoneme boundaries. Second, the signal is RMS-normalised to −20 dBFS under a 0.97 peak ceiling, giving every speaker the same effective loudness without clipping. Third, 0.8 s of very-low-amplitude noise is prepended so the streaming decoder's internal state settles before the first phoneme arrives, and 2 s of trailing silence flushes the final words out of the stream.

Figure 2 shows the chain acting on one of the author's recordings, and Figure 3 shows why it matters: on raw audio the decoder truncates sentence onsets ("THE RESTAURANTS AND SHOPS") and mis-hears "plane" as "PLAYING"; on preprocessed audio both sentences are recovered. The chain raises signal-to-noise ratio and level consistency; what it cannot do is repair vocabulary-level confusions, as the next section shows.

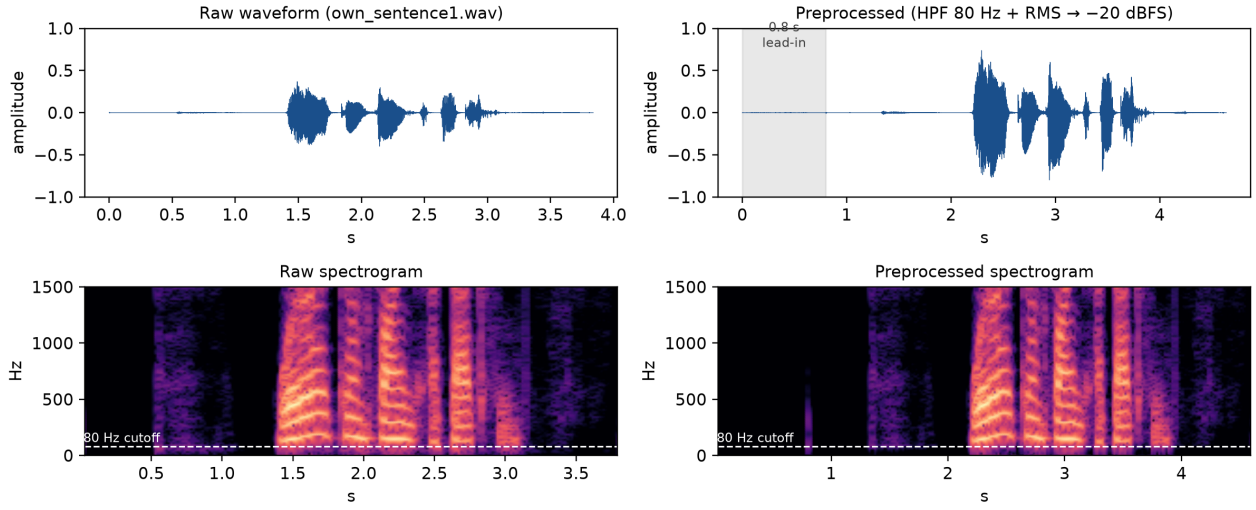


Figure 2: The preprocessing chain applied to `own_sentence1.wav` (16 kHz laptop recording). Top row: RMS normalisation raises speech to −20 dBFS within a 0.97 peak ceiling; the shaded band is the 0.8 s lead-in pad. Bottom row: the zero-phase order-4 high-pass removes all energy below 80 Hz (dashed line) while the speech band is untouched.

```
┌─ EN · Ex4_files/Ex4_audio_files/EN/what_time.wav
│ raw   WHAT TIME IS MY PLAYING
│ clean AT WHAT TIME IS MY PLANE
└──────────────────────────────────────────────────────────────

┌─ EN · Ex4_files/Ex4_audio_files/EN/where.wav
│ raw   THE RESTAURANTS AND SHOPS
│ clean WHERE ARE THE RESTAURANTS AND SHOPS
└──────────────────────────────────────────────────────────────
```

Figure 3: Raw versus preprocessed decodes of two client files: preprocessing recovers the truncated onset of `where.wav` and repairs `what_time.wav`.

Test results and analysis

Every file is decoded after preprocessing and compared with the client's reference transcription. Hypothesis and reference are normalised first (lower-cased, accents and punctuation stripped) so that formatting differences do not count as errors; a word-level Levenshtein alignment then yields substitutions S, deletions D and insertions I over N reference words.

Table 1: Per-file word error rate after preprocessing, computed as WER $= \frac{S+D+I}{N} \times 100$. Corpus WER: English 14.7 %, Spanish 0.0 %, Italian 0.0 %.

Language	File	WER
English	checkin.wav	33.3 %
English	parents.wav	0 %
English	suitcase.wav	33.3 %
English	what_time.wav	20.0 %
English	where.wav	0 %
Spanish	checkin_es.wav	0 %
Spanish	parents_es.wav	0 %
Spanish	suitcase_es.wav	0 %
Spanish	what_time_es.wav	0 %
Spanish	where_es.wav	0 %
Italian	checkin_it.wav	0 %
Italian	parents_it.wav	0 %
Italian	suitcase_it.wav	0 %
Italian	what_time_it.wav	0 %
Italian	where_it.wav	0 %
English	own_sentence1.wav	0 %
Spanish	own_sentence2.wav	0 %

Spanish and Italian are recognised perfectly. English accounts for every remaining error (corpus WER 14.7 %), and inspection of Figure 5 shows the errors are near-homophone and tokenisation failures rather than noise: "check-in desk" decodes as "CHICKEN DESK" (one substitution plus one deletion once the hyphen is split), "suitcase" splits into

"SUIT CASE", and one utterance gains a leading "AT". Preprocessing cannot fix these errors, which are vocabulary and segmentation limits of the compact streaming English model, but it did eliminate every onset-truncation error in the corpus.

The two personal recordings were made on a laptop microphone. Figure 4 shows the raw decode of `own_sentence1.wav` mis-hearing three words ("WERE I CONFINED A TAXI"); after preprocessing, the transcription is exact, and both personal recordings finish at 0 % WER (Figure 5).

```
┌─ EN · Ex4_files/Ex4_audio_files/OWN/own_sentence1.wav
│ raw   WERE I CONFINED A TAXI
│ clean WHERE I CAN FIND A TAXI
└──────────────────────────────────────────────────────────

┌─ ES · Ex4_files/Ex4_audio_files/OWN/own_sentence2.wav
│ raw   Necesito cambiar mi asiento
│ clean Necesito cambiar mi asiento
└──────────────────────────────────────────────────────────
```

Figure 4: The author's two recordings: preprocessing turns a three-word mis-hearing of `own_sentence1.wav` into an exact transcription.

```
EN   checkin.wav           S=1 D=1 I=0 N=6  WER= 33.3%
EN   parents.wav           S=0 D=0 I=0 N=5  WER=  0.0%
EN   suitcase.wav          S=1 D=0 I=1 N=6  WER= 33.3%
EN   what_time.wav         S=0 D=0 I=1 N=5  WER= 20.0%
EN   where.wav             S=0 D=0 I=0 N=6  WER=  0.0%
ES   checkin_es.wav        S=0 D=0 I=0 N=4  WER=  0.0%
ES   parents_es.wav        S=0 D=0 I=0 N=5  WER=  0.0%
ES   suitcase_es.wav       S=0 D=0 I=0 N=6  WER=  0.0%
ES   what_time_es.wav      S=0 D=0 I=0 N=6  WER=  0.0%
ES   where_es.wav          S=0 D=0 I=0 N=7  WER=  0.0%
IT   checkin_it.wav        S=0 D=0 I=0 N=4  WER=  0.0%
IT   parents_it.wav        S=0 D=0 I=0 N=5  WER=  0.0%
IT   suitcase_it.wav       S=0 D=0 I=0 N=7  WER=  0.0%
IT   what_time_it.wav      S=0 D=0 I=0 N=7  WER=  0.0%
IT   where_it.wav          S=0 D=0 I=0 N=7  WER=  0.0%
EN   own_sentence1.wav     S=0 D=0 I=0 N=6  WER=  0.0%
ES   own_sentence2.wav     S=0 D=0 I=0 N=4  WER=  0.0%

Per-language corpus WER:
EN    14.7%
ES     0.0%
IT     0.0%
```

Figure 5: Per-file scores and per-language corpus WER as printed by the evaluation notebook.

ASR in e-commerce: Alibaba's Paraformer

Alibaba's shopping platforms have run customer service on speech for years: by 2017, the AliMe assistant was already serving millions of voice- and text-based customer questions per day[5]. The current generation of Alibaba's speech services is built on **Paraformer**[6], a non-autoregressive model introduced in 2022 that emits the whole token sequence in a

[5] F.-L. Li et al., "AliMe Assist: An intelligent assistant for creating an innovative e-commerce experience," Proc. CIKM, 2017. doi:10.1145/3132847.3133169.

[6] Z. Gao, S. Zhang, I. McLoughlin and Z. Yan, "Paraformer: Fast and accurate parallel transformer for non-autoregressive end-to-end speech recognition," Proc. INTERSPEECH, 2022. arXiv:2206.08317.

single parallel pass: a continuous integrate-and-fire predictor estimates how many tokens the audio contains, and a glancing language model sampler captures the dependencies between them. This yields accuracy comparable to an autoregressive transformer with more than 10× faster decoding, the property that matters when one GPU fleet must transcribe a contact centre. The production model, trained on 60,000 h of industrial Mandarin, is released through the FunASR toolkit together with voice-activity detection and punctuation models[7], and the SeACo extension adds hotword customisation so merchants' product and brand names decode correctly[8].

The contrast with the kiosk prototype is instructive: both are end-to-end neural systems deployed as quantised inference graphs, but the airport assistant needs low-latency word-by-word decoding on a single offline CPU (a streaming transducer), whereas Alibaba optimises server-side throughput over millions of utterances (non-autoregressive batch decoding). Its hotword mechanism is also exactly what the prototype's English model lacks for terms like "check-in".

[7] Z. Gao et al., "FunASR: A fundamental end-to-end speech recognition toolkit," Proc. INTERSPEECH, 2023. arXiv:2305.11013.

[8] X. Shi et al., "SeACo-Paraformer: A non-autoregressive ASR system with flexible and effective hotword customization ability," Proc. ICASSP, 2024. arXiv:2308.03266.

Appendix: acoustic model details

Model bundles (encoder/decoder/joiner + tokens, int8 ONNX):

```
EN  sherpa-onnx-streaming-zipformer
    -en-2023-06-26 (k2-fsa releases)
ES  kroko-es, variant kroko_64l
IT  kroko-it, variant kroko_64l
    (huggingface.co/hudaiapa88)
```

All three bundles are streaming neural transducers with the same three-part layout. The encoder converts log-mel filter-bank frames into acoustic states while consuming the audio in fixed chunks with bounded left context, which is what makes partial results available while the speaker is still talking. The decoder is a small stateless predictor that embeds the most recently emitted tokens, and the joiner combines encoder and decoder states to score the next token; greedy search keeps the best-scoring token frame by frame.

The English encoder is a Zipformer, a U-Net-like refinement of the Conformer that processes the sequence at several frame rates; its file names record the streaming configuration (16-frame chunks, 128 frames of left context) and its token table holds byte-pair units (`bpe.model`). The Spanish and Italian Kroko bundles ship the identical encoder/decoder/joiner layout with language-specific token tables and load through the same `OnlineRecognizer` API.